Инструкция проверяющим и чек-лист D

Собрано из файлов проекта командой `.venv/bin/python -m src.human_docs` · коммит ac61927

Вы оцениваете ответы языковых моделей на задачи по классическому ML. Названия моделей и компаний из ответов убраны и заменены на `[название скрыто]`. Проверяющих двое, каждый работает со своим файлом:

- проверяющий 1 — `grading/sheet_r1.csv`;
- проверяющий 2 — `grading/sheet_r2.csv`.

Строки в файлах одинаковые.

Главные правила

1. **Работайте независимо.** Не обсуждайте оценки друг с другом, пока оба листа не заполнены. Не смотрите чужой лист. По расхождениям двух проверяющих считается каппа Коэна, поэтому обсуждение заранее испортит результат.
2. **Не открывайте `grading/key_DO_NOT_OPEN.csv`.** В нём указано, какая модель дала каждый ответ. Не ищите ответы в `logs/` и `results/` по тексту.
3. **Не пытайтесь угадать модель по стилю.** Оценивайте только содержание по рубрике.
4. **Не поручайте оценку LLM**, ни целиком, ни частично (вопрос самоконтроля № 3).
5. Сомневаетесь — ставьте оценку по рубрике и пишите сомнение в `comment`.

Как открыть и сохранить лист

- Это CSV в кодировке UTF-8, разделитель — запятая. В Google Таблицах: «Файл → Импорт». В LibreOffice: открыть, указать UTF-8 и запятую. В Excel: «Данные → Из текста/CSV», UTF-8.
- Не меняйте порядок и названия столбцов, не удаляйте строки, не меняйте `item_id` и текст в `answer_text`.
- Сохраните результат под тем же именем в формате CSV. Подходит разделитель «,» или «;» и кодировка UTF-8 или Windows-1251: программа поймёт оба варианта. Перед сохранением сделайте копию.
- Ключ `key_DO_NOT_OPEN.csv` лежит в той же папке. Удобнее, если автор работы выдаст каждому проверяющему только его лист и эту инструкцию.

Что заполнять

Заполните **все пустые клетки** в столбцах оценок. Клетки с «—» к строке не относятся, оставьте их как есть. Не ставьте «—» в пустую клетку: если пункт неприменим, пишите «н/п». В `comment` можно писать что угодно.

Столбец	Строки	Значения
`score_0_2`	B, C	0, 1 или 2 по рубрике в столбце `rubric`
`d1_…` – `d6_…`	D	**1** — ошибка есть, **0** — ошибки нет, **н/п** — пункт неприменим
`d_result_correct`	D, если клетка пустая	**1** — код печатает верную итоговую метрику (в пределах допуска из `reference`), **0** — нет

В столбце `human_agrees` листа `parser_spotcheck.csv` пишется только «да» или «нет».

Остальные столбцы нужны для справки:

- `task_text` — условие, которое получила модель;
- `reference` — эталон;
- `rubric` — рубрика или чек-лист;
- `answer_text` — ответ модели;
- `exec_result` — результат запуска кода D без правок;
- `auto_metric` — автопроверка метрики D.

Тип B — обоснование утверждения «верно / неверно»

Сам вердикт уже проверен автоматически. Вы оцениваете вердикт **вместе с обоснованием** по рубрике задачи:

- **2** — вердикт верный, обоснование содержит ключевые аргументы и не содержит грубых ошибок;
- **1** — вердикт верный, но обоснование неполное или с ошибкой;
- **0** — вердикт неверный, обоснования нет или оно неверное по существу.

Если в рубрике задачи (`score_2` / `score_1` / `score_0`) написано конкретнее, действует рубрика.

Тип C — выбор метода

- **2** — приемлемый метод и **все** обязательные аргументы из рубрики;
- **1** — приемлемый метод, но пропущен обязательный аргумент; или спорный метод с верными аргументами;
- **0** — неприемлемый метод или нет обоснования.

Дополнительные аргументы (bonus) балл не повышают выше 2 и не обязательны. Метод, которого нет в списке приемлемых, оценивайте по существу и поясните оценку в `comment`.

Тип D — код scikit-learn

Код был запущен **без правок**, результат лежит в `exec_result`. По каждому пункту чек-листа (`data/checklist_D.md`, там же примеры) поставьте 1, 0 или н/п:

Столбец	Ошибка
`d1_preprocessing_before_split`	масштабирование или другой препроцессинг до разбиения (вне Pipeline)
`d2_tuning_on_test`	подбор гиперпараметров по тестовой выборке
`d3_eval_on_train`	итоговая оценка на обучающей выборке
`d4_accuracy_imbalance`	accuracy при сильном дисбалансе (в том числе scoring по умолчанию)
`d5_no_stratify`	нет стратификации в классификации
`d6_no_random_state`	не зафиксирован random_state там, где есть случайность

- Пункт отмечается, только если ошибка **видна в коде**. Сомнение трактуется в пользу кода, само сомнение опишите в `comment`.
- Если код упал, пункты всё равно оцениваются по тексту кода.

- Если кода в ответе нет, ставьте **н/п** по всем пунктам (оценивать нечего), а в `comment` пишите «кода нет».
- Ошибки вне списка (например, утечка целевой переменной) запишите в `comment`.
- `d_result_correct` заполняйте, если клетка пустая. Сравните **итоговую** метрику из `exec_result` с `reference`. Если код упал, ничего не напечатал или кода нет, ставьте 0.
    - Автопроверка берёт число из последней строки вывода, где есть число. Поэтому она может ошибиться: например, взять `{'C': 10}` вместо accuracy. Её мнение в `auto_metric` — только подсказка.
    - Клетка «—» значит, что автопроверка нашла метрику в допуске («верно (авто)»). Если вы с этим не согласны, напишите об этом в `comment`.

Когда закончили

Сообщите автору работы. Полноту обоих листов проверяет команда `.venv/bin/python -m src.checks g2`. Если каппа Коэна окажется ниже 0,6, рубрику уточнят, и оценку придётся повторить.

Чек-лист методологических ошибок для типа D

Источник: задание ЛР3, раздел «Метрики оценки → Код (тип D)». Проверяют **два человека независимо и вслепую** (листы готовит шаг 10). Модель-автор кода проверяющему не видна.

Код из ответа модели запускается **без правок** (`src/exec_code.py`, шаг 05). Отдельно от чек-листа фиксируются:

- **pass@1** — программа выполнилась без ошибок с первой попытки;
- **корректность результата** — напечатанная метрика отличается от `reference_metric` задачи не больше чем на `tolerance` (относительный допуск).

Для каждой программы по каждому пункту ставится **1** (ошибка есть), **0** (ошибки нет) или **н/п** (пункт неприменим, например стратификация в регрессии).

#	Ошибка	Как выглядит в коде	Пример исправления
1	**Масштабирование (или другой препроцессинг) до разбиения**	`StandardScaler().fit_transform(X)` / `PCA().fit(X)` до `train_test_split` или вне `Pipeline` при кросс-валидации: статистики теста утекают в обучение	`make_pipeline(StandardScaler(), SVC())`, `fit` только на обучающей части
2	**Подбор гиперпараметров по тесту**	цикл по C/k/`max_depth` с выбором лучшего по `score(X_test, y_test)`; `GridSearchCV(...).fit(X, y)` на всех данных, а затем оценка на части тех же данных	`GridSearchCV` только на `X_train`, тест используется один раз в конце
3	**Оценка на обучающей выборке**	итоговая метрика считается на `X_train` (или на всех данных, на которых обучали)	метрика на отложенном `X_test` или на кросс-валидации
4	**Accuracy при сильном дисбалансе**	при доле редкого класса ≤ 10% итоговая метрика или `scoring` для подбора — accuracy (в т. ч. неявно: `scoring` по умолчанию в `GridSearchCV`/`cross_val_score`)	F1 / recall редкого класса, balanced accuracy, ROC AUC, PR AUC; `scoring=` задан явно
5	**Нет стратификации**	классификация (особенно несбалансированная) без `stratify=y` в `train_test_split`; `KFold` без перемешивания на упорядоченных данных	`train_test_split(..., stratify=y)`, `StratifiedKFold(shuffle=True, random_state=...)`
6	**Не зафиксирован random_state**	случайность без seed: `train_test_split`, `KFold(shuffle=True)`, `RandomForest`, `KMeans`, `make_classification` без `random_state` (кроме случаев, когда его задаёт условие)	`random_state=42` везде, где есть случайность

Правила оценки:

- Пункт отмечается, только если ошибка **видна в коде**. Сомнение трактуется в пользу кода, комментарий пишется в поле «примечание».
- Если программа упала до места, где ошибка проявилась бы, пункт всё равно оценивается по коду.

- **Нет масштабирования для метода на расстояниях** (kNN, SVM с RBF-ядром, KMeans) — обязательно записать в «примечание» (решение 15.10.2026; пункты 1–6 взяты из задания и не меняются). Корректность метрики такие решения обычно и так отсеивает (D4: без скейлера 0,71–0,78 при эталоне 0,9556).
- Ошибки, которых нет в списке (например, утечка целевой переменной через признак), записываются в «примечание»: в метрику доли методологических ошибок они не входят, но пригодятся для разбора ошибок.
- **Доля методологических ошибок** по модели = доля программ, где отмечен хотя бы один пункт 1–6 (дополнительно: среднее число пунктов на программу).

Связь с задачами банка: в `notes` у D1–D3 перечислены ловушки, на которые задача рассчитана (например, D3: подбор по accuracy при 5% редкого класса).